Chapter 3 | Worked solutions with Python

Learning to Align

These solutions cover all fourteen exercises in the chapter, from naming the objects in a read to tracing how a loss changes its scores. Keep three questions separate: what is stored, how much of each stored vector is read, and which output token is predicted. A source-position weight is not an output-word probability.

The examples use small teaching arrays and the canonical color-reversal model. Calculations keep full precision until display. The written PDF contains every explanation; the notebook adds executable checks and recorded outputs.

Open this notebook inside the companion repository with the packages in `requirements-ch03.txt` and an IPython kernel. Restart and run all cells in order. Exercise 12 trains for 750 updates on one CPU thread in a temporary directory, then reloads the checkpoint selected by validation loss. Exercise 13 uses that run. The recorded experiment uses Python 3.12.14, PyTorch 2.15.0, and NumPy 2.5.4; small numerical differences across environments are possible.

In [1]

```
from pathlib import Path
import sys
import numpy as np
import torch

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    if (repo_root / "examples/ch03_alignment/model.py").is_file():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError("Open this notebook inside the repository.")

from examples.ch02_recurrence.data import BOS, EOS, PAD, VOCAB, collate, make_splits, words
from examples.ch03_alignment.model import AttentiveSeq2Seq, additive_scores, weighted_read, greedy_decode

torch.set_num_threads(1)
np.set_printoptions(precision=9, suppress=True)
```

1. Distinguish the objects

An embedding is the vector retrieved from the input table for a token ID. The forward encoder then transforms that vector using its preceding states. At *red*, the annotation can depend on *the* and *red*. It has not processed *car*. Although the complete source is available before decoding, the forward annotation at position 2 was computed from the prefix ending there.

Object	Meaning at one decoder step	Size in this example
Annotation $\mathbf{h}_j$	A contextual vector stored at source position j	2 coordinates
Score e_j	Compatibility value before normalization	1 scalar per source position
Weight α_j	Fraction of the source read assigned to position j	1 scalar per source position
Context $\mathbf{r}$	Weighted average of annotations	2 coordinates

The complete attention row has three entries, while the context has two. Their sizes count different things: source positions and vector coordinates. The annotations stay fixed during decoding of this source with fixed weights; the query, score row, weights, and context can change between target steps.

A weight of 0.85 on *car* scales its annotation by 0.85. It is not an 85% probability of predicting *coche*. The context still enters a recurrent update and an output projection. A second softmax, over the target vocabulary, produces output-token probabilities.

In [2]

```
notes = np.array([[1., 0.], [0., 1.], [1., 1.]])
read_weights, read_context = weighted_read(np.log([.05, .10, .85]), notes)
print("Stored annotations:", notes.shape)
print("One attention row:", read_weights.shape)
print("One context:", read_context.shape)
assert read_weights.shape == (3,) and read_context.shape == (2,)
print("The vocabulary distribution requires the decoder and output layer.")
```

Output

```
Stored annotations: (3, 2)
One attention row: (3,)
One context: (2,)
The vocabulary distribution requires the decoder and output layer.
```

2. Read one row

Multiply every coordinate of a vector by the same position weight, then add matching coordinates.

Source vector	Weight for *coche*	Weighted contribution	Weight for *rojo*	Weighted contribution
$(1, 0)$	0.05	$(0.05, 0)$	0.05	$(0.05, 0)$
$(0, 1)$	0.10	$(0, 0.10)$	0.85	$(0, 0.85)$
$(1, 1)$	0.85	$(0.85, 0.85)$	0.10	$(0.10, 0.10)$

The first row of weights gives $(0.90, 0.95)$ and the second gives $(0.15, 0.95)$. The source annotations stay fixed; only the weights change. The second coordinate stays at 0.95 because the last two vectors both have second coordinate 1 and keep the same combined weight.

Every source coordinate is between 0 and 1. A nonnegative weighted average with total weight one must stay in this interval coordinate by coordinate. Thus $(1.2, 0.95)$ is impossible here. The weights sum to one; the context coordinates need not. Context is a vector of features, not a probability distribution over coordinates.

In [3]

```
annotations = np.array([[1., 0.], [0., 1.], [1., 1.]])
for label, weights_row in [("coche", [.05, .10, .85]), ("rojo", [.05, .85, .10])]:
    contributions = np.array(weights_row)[:, None] * annotations
    context = contributions.sum(axis=0)
    assert np.all(context >= annotations.min(axis=0))
    assert np.all(context <= annotations.max(axis=0))
    print(label, "contributions:", contributions.tolist(), "context:", context)
assert not np.all(np.array([1.2, .95]) <= annotations.max(axis=0))
```

Output

```
coche contributions: [[0.05, 0.0], [0.0, 0.1], [0.85, 0.85]] context: [0.9  0.95]
rojo contributions: [[0.05, 0.0], [0.0, 0.85], [0.1, 0.1]] context: [0.15 0.95]
```

3. Recover weights from scores

Imagine that one decoder step wants to give most of its attention to *car* in the source *the red car*. The three source positions receive scores $\ln(0.05)$, $\ln(0.10)$, and $\ln(0.85)$. Each score is the natural logarithm of a positive number. Exponentiation recovers that number. Softmax divides each recovered number by the sum over the three source positions.

Source position	Score	Exponential	Weight
1, *the*	$\ln(0.05) \approx -2.995732$	0.05	0.05
2, *red*	$\ln(0.10) \approx -2.302585$	0.10	0.10
3, *car*	$\ln(0.85) \approx -0.162519$	0.85	0.85

The denominator is $0.05 + 0.10 + 0.85 = 1$. In this example the exponentials already sum to one, so dividing leaves them as they are. We use the logarithms of the original numbers in code and round only the printed scores. Both arrays have shape `(3,)`. Their single axis, also the final axis, identifies the source position. There is one value per position for this decoder step.

In [4]

```
annotations = np.array([[1.0, 0.0], [0.0, 1.0], [1.0, 1.0]])
expected_weights = np.array([0.05, 0.10, 0.85])

scores = np.log(expected_weights)
exponentials = np.exp(scores)
weights = exponentials / exponentials.sum(axis=-1)

assert scores.shape == weights.shape == (3,)
np.testing.assert_allclose(weights, expected_weights)
print("Scores:", np.round(scores, 6), "shape", scores.shape)
print("Exponentials:", exponentials)
print("Weights:", weights, "shape", weights.shape)
```

Output

```
Scores: [-2.995732 -2.302585 -0.162519] shape (3,)
Exponentials: [0.05 0.1  0.85]
Weights: [0.05 0.1  0.85] shape (3,)
```

Adding a constant C to every score multiplies every exponential by the same factor $\exp(C)$. It appears once in the numerator and once in the denominator, so it cancels. For example, the relative weight of *car* and *the* is still $0.85/0.05 = 17$.

$$\frac{\exp(e_j + C)}{\sum_k \exp(e_k + C)} = \frac{\exp(C)\exp(e_j)}{\exp(C)\sum_k \exp(e_k)} = \frac{\exp(e_j)}{\sum_k \exp(e_k)}.$$

This equality uses real arithmetic. When $C = 1000$, the actual shifted scores are around 997 to 1000. Taking their exponentials directly overflows the usual `float64` range. The resulting infinities cannot be divided by their sum to produce valid probabilities.

For a stable softmax, subtract the largest score in the row before taking exponentials. The largest shifted difference is then zero, and every other difference is nonpositive. The largest exponential is therefore one. Subtracting the maximum is a common shift too, so it preserves the mathematical result.

In [5]

```
shifted_scores = scores + 1000.0
with np.errstate(over="ignore"):
    direct_exponentials = np.exp(shifted_scores)

centered = shifted_scores - shifted_scores.max(axis=-1)
safe_exponentials = np.exp(centered)
stable_weights = safe_exponentials / safe_exponentials.sum(axis=-1)

canonical_weights, _ = weighted_read(shifted_scores, annotations)
assert np.isinf(direct_exponentials).all()
assert shifted_scores.shape == stable_weights.shape == (3,)
np.testing.assert_allclose(stable_weights, weights, atol=1e-12, rtol=0)
np.testing.assert_allclose(stable_weights, canonical_weights)
print("Direct exponentials after adding 1000:", direct_exponentials)
print("Centered scores:", np.round(centered, 6))
print("Stable weights:", np.round(stable_weights, 6))
print("Maximum weight change:", f"{np.max(abs(stable_weights-weights)):.2e}")
```

Output

```
Direct exponentials after adding 1000: [inf inf inf]
Centered scores: [-2.833213 -2.140066  0.      ]
Stable weights: [0.05 0.1  0.85]
Maximum weight change: 2.84e-15
```

The tiny difference is from rounding in the computer's representation of the shifted scores. The stable weights agree with the direct calculation on the original scores and with `weighted_read`, the chapter's helper for normalization and the weighted sum.

4. Check the normalization axis

Normalize each target-step row over its source positions. The four row sums are 1. In this invented map, each of the three column sums happens to be $4/3$; they are not probability distributions over target steps. Their total is 4 because the map contains four separate reads.

Attention is not a one-to-one assignment. A position can receive weight at several target steps, and it need not be the largest entry in any row. A second valid map could repeat $(0.9, 0.05, 0.05)$ at every step: positions 2 and 3 would contribute to every read without ever having the largest weight. Reusing an annotation does not consume it.

In [6]

```
teaching_map = np.array([[.90, .05, .05], [.05, .10, .85],
                         [.05, .85, .10], [1/3, 1/3, 1/3]])
np.testing.assert_allclose(teaching_map.sum(axis=1), 1.)
np.testing.assert_allclose(teaching_map.sum(axis=0), 4/3)
print("Row sums:", teaching_map.sum(axis=1))
print("Column sums:", teaching_map.sum(axis=0))
print("Total weight across four reads:", teaching_map.sum())
```

Output

```
Row sums: [1. 1. 1. 1.]
Column sums: [1.333333333 1.333333333 1.333333333]
Total weight across four reads: 4.000000000000001
```

5. Let padding steal a read

The original exponentials sum to 1. Adding a padding score of 0 adds $\exp(0) = 1$ to the denominator, making it 2. The unmasked row is therefore $(0.025, 0.05, 0.425, 0.5)$. Its last vector is zero, so the resulting read is

$$\mathbf{r} = 0.025(1, 0) + 0.05(0, 1) + 0.425(1, 1) + 0.5(0, 0) = (0.45, 0.475).$$

Zeroing the padding vector removes its contribution to the sum but not its share of the normalization. Masking its score before softmax makes that weight zero and restores the real weights $(0.05, 0.10, 0.85)$ and read $(0.90, 0.95)$. Packing and source masking solve different problems: packing excludes padding from the recurrent encoder updates; masking excludes padded annotations from each attention read.

In [7]

```
padded_annotations = np.vstack((annotations, [0., 0.]))
padded_scores = np.r_[np.log([.05, .10, .85]), 0.]
unmasked_weights, unmasked_context = weighted_read(padded_scores, padded_annotations)
masked_weights, masked_context = weighted_read(
    padded_scores, padded_annotations, np.array([True, True, True, False]))
np.testing.assert_allclose(unmasked_weights, [.025, .05, .425, .5])
np.testing.assert_allclose(unmasked_context, [.45, .475])
np.testing.assert_allclose(masked_weights, [.05, .10, .85, 0.])
np.testing.assert_allclose(masked_context, [.90, .95])
print("Unmasked:", unmasked_weights, "read:", unmasked_context)
print("Masked:  ", masked_weights, "read:", masked_context)
```

Output

```
Unmasked: [0.025 0.05  0.425 0.5  ] read: [0.45  0.475]
Masked:   [0.05 0.1  0.85 0.  ] read: [0.9  0.95]
```

6. Open the scorer

Keep the annotations $(1, 0)$, $(0, 1)$, and $(1, 1)$ in source order. The decoder's previous hidden state supplies the query $\mathbf{q} = (0.2, -0.1)$. Both projection matrices are identities and the bias is zero. For one annotation, add the

query coordinate by coordinate, apply $\tanh$ to each sum, then multiply the resulting coordinates by 1 and -1 and add. The direction $\mathbf{v}_a = (1, -1)^\mathsf{T}$ therefore takes the first transformed coordinate minus the second.

$$e_1 = \tanh(1.2) - \tanh(-0.1) \approx 0.833654607012 + 0.099667994625 \approx 0.933322601637.$$

The other two rows apply the same rule to different annotations. Use the full calculator values for the exponential and softmax.

Position	**Sum before** $\tanh$	**Score calculation**	**Displayed score**
the	$(1.2, -0.1)$	$\tanh(1.2) - \tanh(-0.1)$	0.933323
red	$(0.2, 0.9)$	$\tanh(0.2) - \tanh(0.9)$	-0.518923
car	$(1.2, 0.9)$	$\tanh(1.2) - \tanh(0.9)$	0.117357

The unshifted exponentials are about 2.542944, 0.595161, and 1.124521. Divide by their sum, about 4.262626. This gives $\boldsymbol{\alpha} \approx (0.596568, 0.139623, 0.263809)$. The read uses the original annotations. Only positions 1 and 3 contribute to the first coordinate and only 2 and 3 to the second coordinate. Before rounding, the read is $(\alpha_1 + \alpha_3, \alpha_2 + \alpha_3)$, giving $\mathbf{r} \approx (0.860377, 0.403432)$.

The direction is a two-coordinate vector. In NumPy, the multiplication of a `(3, 2)` feature matrix by a direction of shape `(2,)` gives `(3,)`. Multiplication by a direction of shape `(2, 1)` gives the same scalars in a `(3, 1)` array. The chapter's helper accepts either form and removes the singleton axis from the direction before multiplying. The helper returns `(3,)` in both cases. Normalizing the trailing axis of a raw `(3, 1)` result would normalize three separate singletons, giving three ones.

In [8]

```
identity = np.eye(2)
query = np.array([0.2, -0.1])
direction = np.array([1.0, -1.0])
direction_column = direction.reshape(2, 1)

vector_scores = additive_scores(
    query, annotations, identity, identity, direction
)
column_scores = additive_scores(
    query, annotations, identity, identity, direction_column
)
alpha, context = weighted_read(vector_scores, annotations)

expected_scores = [0.933322601637, -0.518922549974, 0.117356736813]
expected_alpha = [0.596567504376, 0.139623183989, 0.263809311635]
expected_context = [0.860376816011, 0.403432495624]
assert vector_scores.shape == column_scores.shape == (3,)
np.testing.assert_array_equal(vector_scores, column_scores)
np.testing.assert_allclose(vector_scores, expected_scores, atol=1e-12)
np.testing.assert_allclose(alpha, expected_alpha, atol=1e-12)
np.testing.assert_allclose(context, expected_context, atol=1e-12)

print("Direction shapes:", direction.shape, direction_column.shape)
print("Score shapes:", vector_scores.shape, column_scores.shape)
print("Scores:", np.round(vector_scores, 6))
print("Weights:", np.round(alpha, 6))
print("Read:", np.round(context, 6))
```

Output

```
Direction shapes: (2,) (2, 1)
Score shapes: (3,) (3,)
Scores: [ 0.933323 -0.518923  0.117357]
Weights: [0.596568 0.139623 0.263809]
Read: [0.860377 0.403432]
```

Change only the query to $(-0.2, 0.1)$. For *the*, the sum is now $(0.8, 0.1)$, so its score is $\tanh(0.8) - \tanh(0.1) \approx 0.564369$. Repeat for the remaining positions and divide the new exponentials by their sum, about 2.999439.

Position	**Sum before** $\tanh$	**New score**	**Old weight**	**New weight**
the	$(0.8, 0.1)$	0.564369	0.596568	0.586222
red	$(-0.2, 1.1)$	-0.997874	0.139623	0.122910
car	$(0.8, 1.1)$	-0.136462	0.263809	0.290867

Using the new, unrounded weights gives

$$\mathbf{r}' = (\alpha'_1 + \alpha'_3, \alpha'_2 + \alpha'_3) \approx (0.877090, 0.413778).$$

Round only after adding the contributions.

In [9]

```
new_query = np.array([-0.2, 0.1])
new_scores = additive_scores(
    new_query, annotations, identity, identity, direction
)
new_alpha, new_context = weighted_read(new_scores, annotations)
np.testing.assert_allclose(
    new_alpha, [0.586222124962, 0.122910402253, 0.290867472785]
)
np.testing.assert_allclose(new_context, [0.877089597747, 0.413777875038])
print("New scores:", np.round(new_scores, 6))
print("New weights:", np.round(new_alpha, 6))
print("Weight change:", np.round(new_alpha - alpha, 6))
print("New read:", np.round(new_context, 6))
```

Output

```
New scores: [ 0.564369 -0.997874 -0.136462]
New weights: [0.586222 0.12291  0.290867]
Weight change: [-0.010345 -0.016713  0.027058]
New read: [0.87709  0.413778]
```

The first position still has the largest weight. The weight on *car* increases even though its own raw score falls. Its score falls by less than either of the other two, so it gains weight relative to them. Because the query is added inside the nonlinear transformation, changing it does not generally add the same constant to each score. Exercise 8 shows what happens when the transformation is linear.

7. Follow the shapes

Read axis names as well as sizes. The attention softmax runs along the source axis S; the vocabulary softmax runs along the vocabulary axis V.

Quantity	Shape	Axis meanings
Annotations	$(2, 5, 64)$	batch, source position, hidden feature
Projected annotations	$(2, 5, 32)$	batch, source position, scoring feature
One score or weight row per item	$(2, 5)$	batch, source position
One context	$(2, 64)$	batch, hidden feature
All attention rows	$(2, 6, 5)$	batch, target step, source position
All vocabulary logits	$(2, 6, 9)$	batch, target step, vocabulary class
Each encoder final state	$(1, 2, 64)$	layer times direction, batch, feature
Each decoder cell state	$(2, 64)$	batch, feature
Joined decoder input	$(2, 80)$	batch, 16 embedding + 64 context features

Removing the encoder's one-layer axis changes storage shape, not state values. Hidden and cell states are both transferred. Annotations, their source projections, and the validity mask can be reused across target steps for this source. Queries, scores, weights, contexts, and decoder states are recalculated. During training, the reused projections retain their computation graph so gradients can reach the encoder.

With PyTorch's two bias tensors per recurrent module, the model stores 63,401 parameter entries. This includes 32 entries in two PAD embedding rows fixed at zero. Source length changes the annotation storage, not this parameter count.

In [10]

```
with torch.random.fork_rng(devices=[]):
    torch.manual_seed(31)
    shape_model = AttentiveSeq2Seq().eval()
shape_batch = collate([(3, 4, 5), (6, 4, 3, 5, 8)])
with torch.inference_mode():
    shape_memory, shape_states = shape_model.encode(shape_batch.source, shape_batch.lengths)
    shape_context, shape_weights = shape_model.attention(shape_states[0], *shape_memory)
    shape_logits, shape_maps = shape_model(shape_batch.source, shape_batch.lengths, shape_batch.decoder_input)
print("Annotations, projections, mask:", [tuple(x.shape) for x in shape_memory])
print("Decoder hidden and cell:", [tuple(x.shape) for x in shape_states])
print("Context, weights:", tuple(shape_context.shape), tuple(shape_weights.shape))
print("All weights, logits:", tuple(shape_maps.shape), tuple(shape_logits.shape))
print("Joined input width:", shape_model.decoder.input_size)
assert shape_maps.shape == (2, 6, 5) and shape_logits.shape == (2, 6, 9)
assert sum(p.numel() for p in shape_model.parameters()) == 63401
print("Stored parameter entries: 63401; fixed PAD embedding entries: 32")
```

Output

```
Annotations, projections, mask: [(2, 5, 64), (2, 5, 32), (2, 5)]
Decoder hidden and cell: [(2, 64), (2, 64)]
Context, weights: (2, 64) (2, 5)
All weights, logits: (2, 6, 5) (2, 6, 9)
Joined input width: 80
Stored parameter entries: 63401; fixed PAD embedding entries: 32
```

8. Remove the nonlinearity

Without $\tanh$, distribute the final projection over the sum:

$$e_j = (\mathbf{s}W_s)\mathbf{v}_a + (\mathbf{h}_j W_h + \mathbf{b}_a)\mathbf{v}_a = C(\mathbf{s}) + d_j.$$

The query-dependent value $C(\mathbf{s})$ is the same for every source position. It cancels during normalization:

$$\alpha_j = \frac{\exp(C + d_j)}{\sum_k \exp(C + d_k)} = \frac{\exp(d_j)}{\sum_k \exp(d_k)}.$$

For fixed annotations and mask, changing the query cannot change this simplified scorer's weights. The decoder could still use its state elsewhere; the conclusion concerns this particular linear scoring expression. Adding the query *inside* a nonlinearity permits different score changes at different source positions, so the relative weights can respond to the query.

In [11]

```
linear_reads = []
for request in [np.array([.2, -.1]), np.array([-.2, .1])]:
    linear_scores = (annotations + request) @ np.array([1., -1.])
    linear_weights, linear_context = weighted_read(linear_scores, annotations)
    linear_reads.append(linear_weights)
    print("Linear scores:", linear_scores, "weights:", linear_weights)
np.testing.assert_allclose(linear_reads[0], linear_reads[1], atol=1e-15)
assert not np.allclose(alpha, new_alpha)
print("The nonlinear scorer's weights differ for the two queries.")
```

Output

```
Linear scores: [ 1.3 -0.7  0.3] weights: [0.665240956 0.090030573 0.244728471]
Linear scores: [ 0.7 -1.3 -0.3] weights: [0.665240956 0.090030573 0.244728471]
The nonlinear scorer's weights differ for the two queries.
```

9. Test source padding

A batch holds the sources *red green blue* and *yellow red*. Their true lengths are three and two. The stored width is three, so the second source already ends with one `PAD`. Extend the stored width to six by appending three more `PAD` columns to both sources. The true lengths and decoder input stay fixed. Both runs should use exactly the same real source tokens.

For this check, the model needs a fixed set of weights. A seeded, untrained model in evaluation mode lets us test the computation without involving any learned behavior. CPU float64 keeps the numerical comparison precise. We use absolute tolerance 10^{-12} and relative tolerance 10^{-10} for quantities expected to agree. Different stored widths can affect floating-point reduction order even when the mathematical result agrees.

In [12]

```
from examples.ch02_recurrence.data import PAD, VOCAB, collate
from examples.ch03_alignment.model import AttentiveSeq2Seq

tolerance9 = {"atol": 1e-12, "rtol": 1e-10}
with torch.random.fork_rng(devices=[]):
    torch.manual_seed(31)
    model9 = AttentiveSeq2Seq().double().eval()

batch9 = collate([(3, 4, 5), (6, 3)])
width9 = batch9.source.size(1)
extra9 = batch9.source.new_full((len(batch9.source), 3), PAD)
extended9 = torch.cat((batch9.source, extra9), dim=1)

with torch.inference_mode():
    logits9, weights9 = model9(
        batch9.source, batch9.lengths, batch9.decoder_input
    )
    extended_logits9, extended_weights9 = model9(
        extended9, batch9.lengths, batch9.decoder_input
    )
print("Compared source widths:", batch9.source.size(1), extended9.size(1))
```

Output

```
Compared source widths: 3 6
```

The target labels tell us which decoder steps represent real predictions. Keep every non-PAD label, including EOS. The lengths give four such steps for the first example and three for the second. Compare the nine vocabulary logits at each of those seven steps. For attention, slice the extended tensor back to its original source width. The first three entries of each row should match the original row. The three appended entries should be exactly zero at every step. The second source's old padding entry should still be zero.

In [13]

```
valid_steps9 = batch9.target != PAD
logit_delta9 = (
    extended_logits9[valid_steps9] - logits9[valid_steps9]
).abs().max().item()
kept_weights9 = extended_weights9[..., :width9]
weight_delta9 = (kept_weights9 - weights9).abs().max().item()
new_weights9 = extended_weights9[..., width9:]

torch.testing.assert_close(
    extended_logits9[valid_steps9],
    logits9[valid_steps9],
    **tolerance9,
)
torch.testing.assert_close(kept_weights9, weights9, **tolerance9)
assert torch.count_nonzero(new_weights9).item() == 0
assert torch.count_nonzero(kept_weights9[1, :, 2]).item() == 0

sums9 = extended_weights9[valid_steps9].sum(dim=-1)
torch.testing.assert_close(sums9, torch.ones_like(sums9), **tolerance9)

print("True source lengths:", batch9.lengths.tolist())
print("Stored source widths:", width9, "->", extended9.size(1))
print("Valid prediction steps:", valid_steps9.sum().item())
print(f"Max absolute change in valid logits: {logit_delta9:.2e}")
print(f"Max absolute change in original weights: {weight_delta9:.2e}")
nonzero9 = new_weights9.count_nonzero().item()
print("Nonzero weights in the appended columns:", nonzero9)
```

Output

```
True source lengths: [3, 2]
Stored source widths: 3 -> 6
Valid prediction steps: 7
Max absolute change in valid logits: 0.00e+00
Max absolute change in original weights: 0.00e+00
Nonzero weights in the appended columns: 0
```

Packing the encoder with the true lengths keeps padding out of its recurrent updates. The source-validity mask then assigns a score of $-\infty$ to each stored padding position before softmax. Each such entry adds zero to the denominator. Every real weight and the resulting context therefore agree up to rounding, so the decoder receives the same attention input at each step.

An entirely padded source has no positions over which to form a probability distribution. If all scores were $-\infty$, the sum of their exponentials would be zero. The encoder rejects an empty real source. The attention module also rejects an all-false validity mask if called directly.

In [14]

```
all_pad9 = batch9.source.new_full((1, 3), PAD)
empty_length9 = batch9.lengths.new_tensor([0])
try:
    model9.encode(all_pad9, empty_length9)
except ValueError as error9:
    print(f"Encoder rejected the source: {error9}")
else:
    raise AssertionError("An empty source was accepted.")

with torch.inference_mode():
    memory9, initial9 = model9.encode(batch9.source, batch9.lengths)
    annotations9, projected9, valid_source9 = memory9
    all_masked9 = torch.zeros_like(valid_source9)
    try:
        model9.attention(
            initial9[0], annotations9, projected9, all_masked9
        )
    except ValueError as error9:
        print(f"Attention rejected the mask: {error9}")
    else:
        raise AssertionError("An all-false source mask was accepted.")
```

Output

```
Encoder rejected the source: provide a nonempty source batch and valid lengths
Attention rejected the mask: every source needs at least one valid position
```

10. Separate two information rules

In offline translation, the entire input sentence is available before the decoder starts. The encoder can finish reading it and retain one annotation per real source position. At the first target step the decoder can already read all those annotations. In the forward LSTM used here, each annotation contains information from its own source prefix, but annotations for every position have already been computed.

The target has a different timeline. When predicting target y_t, a generating decoder has `BOS` and its previous choices through y_{t-1}. Teacher forcing supplies the corresponding **reference** prefix for the same computation. We can perturb a later teacher-forced input and ask which outputs depend on it.

Use *red green blue black* as the source. Reversal gives a teacher-forced input row of `BOS black blue green red`. Change only input step 3 from *blue* to *yellow*. Keep the original source and all other decoder inputs fixed.

In [15]

```
tolerance10 = {"atol": 1e-12, "rtol": 1e-10}
with torch.random.fork_rng(devices=[]):
    torch.manual_seed(31)
    model10 = AttentiveSeq2Seq().double().eval()

batch10 = collate([(3, 4, 5, 8)])
changed_step10 = 2  # Python index 2 is decoder step 3.
changed_inputs10 = batch10.decoder_input.clone()
changed_inputs10[0, changed_step10] = VOCAB.index("yellow")

with torch.inference_mode():
    logits10, weights10 = model10(
        batch10.source, batch10.lengths, batch10.decoder_input
    )
    changed_logits10, changed_weights10 = model10(
        batch10.source, batch10.lengths, changed_inputs10
    )
print("Original inputs:", words(batch10.decoder_input[0].tolist()))
print("Modified inputs:", words(changed_inputs10[0].tolist()))
```

Output

```
Original inputs: <bos> black blue green red
Modified inputs: <bos> black yellow green red
```

Both runs have identical inputs through step 2. Their logits and attention rows must therefore agree at steps 1 and 2. At step 3, the attention read uses the hidden state from step 2 as its query. It still sees the same query and the same source. The changed input embedding enters the LSTM only after this read. Compare the attention row at step 3 and check that the chosen perturbation changes the step-3 vocabulary logits in this run.

In [16]

```
earlier10 = slice(0, changed_step10)
torch.testing.assert_close(
    changed_logits10[:, earlier10], logits10[:, earlier10], **tolerance10
)
through_changed10 = slice(0, changed_step10 + 1)
torch.testing.assert_close(
    changed_weights10[:, through_changed10],
    weights10[:, through_changed10],
    **tolerance10,
)

logit_deltas10 = (changed_logits10 - logits10).abs().amax(dim=-1)[0]
weight_deltas10 = (changed_weights10 - weights10).abs().amax(dim=-1)[0]
assert logit_deltas10[changed_step10].item() > tolerance10["atol"]
assert weight_deltas10[changed_step10 + 1].item() > tolerance10["atol"]

print("Step  Original  Modified  Max logit delta  Max weight delta")
for index10, (old10, new10) in enumerate(
    zip(batch10.decoder_input[0].tolist(), changed_inputs10[0].tolist())
):
    logit10 = logit_deltas10[index10].item()
    weight10 = weight_deltas10[index10].item()
    print(
        f"{index10 + 1:>4}  {VOCAB[old10]:<8}  {VOCAB[new10]:<8}  "
        f"{logit10:>15.2e}  {weight10:>16.2e}"
    )
```

Output

```
Step  Original  Modified  Max logit delta  Max weight delta
   1  <bos>     <bos>            0.00e+00          0.00e+00
   2  black     black            0.00e+00          0.00e+00
   3  blue      yellow           9.63e-02          0.00e+00
   4  green     green            5.03e-02          7.50e-05
   5  red       red              2.69e-02          2.07e-05
```

The logit change starts at step 3 while the attention change first appears at step 4. At step 3, the LSTM combines the same source context with two different token embeddings. Its two hidden states can differ, so the output layer can assign different logits. Step 4 uses those hidden states as its attention queries. The two runs can now read the same source with different weights, even though they receive the same step-4 input token. The observed differences at steps 3 and 4 are specific to this perturbation and these weights.

11. Reorder stored annotations

Store *red green blue* with one padding slot and encode the source once. Fix the query to the decoder's original initial hidden state. The four source slots hold triples of **annotation**, **projected annotation**, and **validity flag**. Move each triple together so the order is now *blue, PAD, red, green*. We move already computed vectors. Their values and the decoder query stay fixed.

Let $\pi(j)$ be the old index of the row moved to new slot j. The score in the new slot is $s_{\pi(j)}$ because scoring compares that row with the same query. The validity flag follows the row, and softmax still sums over the same valid scores. Thus the new weight is $\alpha_{\pi(j)}$, and the new context is

$$\sum_j \alpha_{\pi(j)}\,\mathbf{h}_{\pi(j)} = \sum_i \alpha_i\,\mathbf{h}_i.$$

The experiment checks both predictions. The new weight vector should be the old vector indexed by the permutation. The context should agree within floating-point tolerance.

In [17]

```
tolerance11 = {"atol": 1e-12, "rtol": 1e-10}
with torch.random.fork_rng(devices=[]):
    torch.manual_seed(31)
    model11 = AttentiveSeq2Seq().double().eval()

batch11 = collate([(3, 4, 5)])
source11 = torch.nn.functional.pad(batch11.source, (0, 1), value=PAD)
order11 = torch.tensor([2, 3, 0, 1])

with torch.inference_mode():
    memory11, state11 = model11.encode(source11, batch11.lengths)
    query11 = state11[0]
    context11, weights11 = model11.attention(query11, *memory11)
    permuted11 = tuple(component[:, order11] for component in memory11)
    moved_context11, moved_weights11 = model11.attention(
        query11, *permuted11
    )

torch.testing.assert_close(
    moved_weights11, weights11[:, order11], **tolerance11
)
torch.testing.assert_close(moved_context11, context11, **tolerance11)
assert moved_weights11[0, 1].item() == 0.0

labels11 = [VOCAB[token] for token in source11[0].tolist()]
moved_labels11 = [labels11[index] for index in order11.tolist()]
print("Old slots" + "".join(f"{label:>10}" for label in labels11))
print("Old read " + "".join(f"{weight:10.6f}" for weight in weights11[0]))
print("New slots" + "".join(f"{label:>10}" for label in moved_labels11))
moved_values11 = "".join(f"{weight:10.6f}" for weight in moved_weights11[0])
print("New read " + moved_values11)
move_delta11 = (moved_context11 - context11).abs().max().item()
print(f"Max absolute context change after moving slots: {move_delta11:.2e}")
```

Output

```
Old slots       red     green      blue     <pad>
Old read   0.323653  0.340577  0.335771  0.000000
New slots      blue     <pad>       red     green
New read   0.335771  0.000000  0.323653  0.340577
Max absolute context change after moving slots: 1.39e-17
```

Now move the **tokens** before encoding. The new source is *blue red green*. Right-pad it again because the encoder uses a valid prefix of the stored tensor. In the old source, the LSTM reaches *blue* after *red green*. In the new source, it reaches *blue* from the initial encoder state. Its annotation can change even though its token embedding is the same.

Put the new annotations and weights into the original token order before comparing them. Keep the original decoder query for the new attention read. This isolates the effect of re-encoding the source. A full decoding pass would also start from the new final encoder state.

In [18]

```
new_source11 = source11[:, [2, 0, 1, 3]]
back_to_old11 = [1, 2, 0, 3]

with torch.inference_mode():
    new_memory11, _ = model11.encode(new_source11, batch11.lengths)
    new_context11, new_weights11 = model11.attention(
        query11, *new_memory11
    )

matched_annotations11 = new_memory11[0][:, back_to_old11]
matched_weights11 = new_weights11[:, back_to_old11]
annotation_delta11 = (
    matched_annotations11 - memory11[0]
).abs().max().item()
weight_delta11 = (matched_weights11 - weights11).abs().max().item()
context_delta11 = (new_context11 - context11).abs().max().item()
assert annotation_delta11 > tolerance11["atol"]
assert context_delta11 > tolerance11["atol"]

print("Re-encoded source: blue red green")
print(f"Max annotation change in matched slots: {annotation_delta11:.2e}")
print(f"Max weight change in matched slots:     {weight_delta11:.2e}")
print(f"Max context change with fixed query:    {context_delta11:.2e}")
```

Output

```
Re-encoded source: blue red green
Max annotation change in matched slots: 1.24e-01
Max weight change in matched slots:     7.81e-04
Max context change with fixed query:    6.44e-02
```

Rearranging the stored rows changes which slot holds each weight. Re-encoding changes the vectors that the decoder reads, because the recurrent encoder computes each vector from the current embedding and the states produced by the preceding tokens.

12. Inspect an actual prediction path

Suppose we reverse six colors. The decoder has to emit six colors in the opposite order, followed by the end token. An attention row records the source read used at one prediction step. We will compare two ways to reach those steps using the same source and the same trained model.

Teacher forcing supplies the reference prefix. The input to step 1 is BOS, the beginning-of-sequence token. The input to step 2 is the first reference color, even if the first prediction differs from it. Each row is labeled by the **reference token** we ask the model to score. The model's highest-scoring choice can differ from that label.

Greedy generation supplies the model's own prefix. It also starts with BOS, then chooses the highest-scoring token at each step and feeds that choice into the next step. Each row is labeled by the **token actually emitted**. Generation ends on EOS, the end-of-sequence token, or at the generation cap.

First train the repository's Chapter 3 color-reversal model for 750 updates on a CPU with seed 2026. The training helper saves the weights from the update with the lowest validation token loss, the mean negative log probability of the valid reference labels. Reload those weights from the checkpoint before tracing any prediction. The temporary directory keeps the exercise independent of earlier runs.

In [19]

```
from contextlib import redirect_stdout
from io import StringIO
from tempfile import TemporaryDirectory

from examples.ch02_recurrence.data import EOS, VOCAB, collate, words
from examples.ch02_recurrence.data import make_splits
from examples.ch03_alignment.model import AttentiveSeq2Seq
from examples.ch03_alignment.model import greedy_decode
from examples.ch03_alignment.run_examples import run_training

with TemporaryDirectory(prefix="ch03-solutions-") as output:
    with redirect_stdout(StringIO()):
        alignment_run = run_training(
            steps=750, seed=2026, output=Path(output)
        )
    checkpoint = torch.load(
        Path(output) / "best.pt", map_location="cpu", weights_only=True
    )

print("Selected update", checkpoint["selected_step"])
validation_loss = alignment_run["validation"]["token_nll"]
print(f"Validation token loss {validation_loss:.9f}")
alignment_env = alignment_run["environment"]
print(f"Python {alignment_env['python']}, PyTorch {alignment_env['torch']}")
print(f"NumPy {alignment_env['numpy']}, CPU, one thread")
```

Output

```
Selected update 750
Validation token loss 0.001809386
Python 3.12.14, PyTorch 2.15.0
NumPy 2.5.4, CPU, one thread
```

Rebuild the model with the widths in the checkpoint. Set it to evaluation mode and turn off gradient recording for the traces. We use the illustration selected by the training helper. It picks a correct six-color test example with as many distinct colors as possible, breaking ties by test order. If no six-color example is correct, it uses the first six-color test source. This is an illustration chosen after evaluation. The validation loss has already determined the checkpoint.

In [20]

```
alignment_config = checkpoint["configuration"]
trained_alignment = AttentiveSeq2Seq(
    embedding_width=alignment_config["embedding_width"],
    hidden_width=alignment_config["hidden_width"],
    alignment_width=alignment_config["alignment_width"],
)
trained_alignment.load_state_dict(checkpoint["model"])
trained_alignment.eval()

_, _, alignment_test = make_splits(seed=alignment_config["seed"])
illustrated = alignment_run["alignment_example"]
alignment_source = alignment_test[illustrated["test_index"]]
alignment_batch = collate([alignment_source])
generation_cap = alignment_config["generation_cap"]

print("Held-out example", illustrated["test_index"] + 1)
print("Source", words(alignment_source))
```

Output

```
Held-out example 12
Source blue blue yellow green white red
```

Give the source and its true length to both passes. Only the teacher-forced call receives the shifted reference inputs, namely BOS followed by the reversed colors. The greedy call receives a cap of seven new tokens. That permits six colors and EOS.

The greedy trace retains the row used to predict the first EOS. It retains every row if no EOS is generated and reports an unfinished sequence. PAD or BOS emitted before EOS would be an actual model choice and would stay in the trace. Here we trace one source at a time, so there are no rows inserted for a finished batch member.

In [21]

```
with torch.inference_mode():
    forced_logits, forced_maps = trained_alignment(
        alignment_batch.source,
        alignment_batch.lengths,
        alignment_batch.decoder_input,
    )
    greedy_tokens, greedy_maps = greedy_decode(
        trained_alignment,
        alignment_batch.source,
        alignment_batch.lengths,
        generation_cap,
    )

reference_ids = alignment_batch.target[0].tolist()
forced_choices = forced_logits[0].argmax(dim=-1).tolist()
greedy_ids = greedy_tokens[0].tolist()
ended = EOS in greedy_ids
if ended:
    greedy_ids = greedy_ids[:greedy_ids.index(EOS) + 1]
else:
    assert len(greedy_ids) == generation_cap

forced_weights = forced_maps[0, :len(reference_ids)]
greedy_weights = greedy_maps[0, :len(greedy_ids)]
for rows in (forced_weights, greedy_weights):
    torch.testing.assert_close(
        rows.sum(dim=-1), torch.ones_like(rows[:, 0])
    )

if greedy_ids == reference_ids:
    torch.testing.assert_close(forced_weights, greedy_weights)

print("Reference       ", words(reference_ids))
print("Teacher argmax  ", " ".join(VOCAB[i] for i in forced_choices))
print("Greedy output   ", words(greedy_ids))
if ended:
    print("EOS at generated step", len(greedy_ids))
else:
    print("Generation cap reached without EOS", generation_cap)
```

Output

```
Reference       red white green yellow blue blue <eos>
Teacher argmax  red white green yellow blue blue <eos>
Greedy output   red white green yellow blue blue <eos>
EOS at generated step 7
```

Each column is one **source position**. Numbering the columns keeps two occurrences of the same color distinct. Each row is a prediction step, with a distribution over all six source positions. The first table uses reference labels and the second uses generated tokens. Both maps include their ending rows in this run. The calculations keep full precision. Values shown as 0.0000 can be positive weights smaller than the display precision.

In [22]

```
def show_alignment(title, label, tokens, weights, source):
    print(title)
    print("Step " + f"{label:<10}" + " ".join(f"{j+1}:{VOCAB[t]:>5}" for j, t in enumerate(source)))
    for step, (token, row) in enumerate(zip(tokens, weights), 1):
        print(f"{step:>4} {VOCAB[token]:<10}" + " ".join(f"{float(x):7.4f}" for x in row))

show_alignment("Teacher-forced read", "Reference", reference_ids, forced_weights, alignment_source)
show_alignment("Greedy read", "Emitted", greedy_ids, greedy_weights, alignment_source)
```

Output

```
Teacher-forced read
Step Reference 1: blue 2: blue 3:yellow 4:green 5:white 6:  red
   1 red        0.0069  0.0026  0.0020  0.0040  0.0794  0.9051
   2 white      0.0001  0.0001  0.0008  0.0813  0.6189  0.2988
   3 green      0.0000  0.0000  0.0485  0.8316  0.0944  0.0255
   4 yellow     0.0000  0.0040  0.9123  0.0654  0.0107  0.0077
   5 blue       0.0016  0.7931  0.1948  0.0060  0.0024  0.0021
   6 blue       0.8443  0.1391  0.0070  0.0044  0.0027  0.0025
   7 <eos>      0.3402  0.1324  0.1416  0.1438  0.1220  0.1199
Greedy read
Step Emitted   1: blue 2: blue 3:yellow 4:green 5:white 6:  red
   1 red        0.0069  0.0026  0.0020  0.0040  0.0794  0.9051
   2 white      0.0001  0.0001  0.0008  0.0813  0.6189  0.2988
   3 green      0.0000  0.0000  0.0485  0.8316  0.0944  0.0255
   4 yellow     0.0000  0.0040  0.9123  0.0654  0.0107  0.0077
   5 blue       0.0016  0.7931  0.1948  0.0060  0.0024  0.0021
   6 blue       0.8443  0.1391  0.0070  0.0044  0.0027  0.0025
   7 <eos>      0.3402  0.1324  0.1416  0.1438  0.1220  0.1199
```

In the saved run, the source is *blue blue yellow green white red*. Both passes produce choices *red white green yellow blue blue EOS*. The largest weights for the first six steps occur at source positions 6, 5, 4, 3, 2, and 1. The EOS row is spread across the source. Reversal specifies a color at every ordinary output step and specifies when to stop.

The two maps match in this example because every greedy token matches the reference. The decoder receives the same input tokens in the same order in both passes, so the previous hidden state used to calculate each read matches as well. In a trace with a wrong generated token, that choice becomes the next greedy input. The attention row at that next step still uses the hidden state before the new input is consumed. Once different inputs lead to different hidden states, later attention rows can differ. The row labels record how the two traces were obtained even when all values match.

The measured greedy read in the recorded run is shown below. The teacher-forced read agrees within numerical tolerance because the generated prefix matches the reference at every step. The selected update was 750; validation and test exact match were both 48/48. These are results on the supplied lengths, not on longer unseen lengths.

Step / emitted	1 blue	2 blue	3 yellow	4 green	5 white	6 red
1 red	0.0069	0.0026	0.0020	0.0040	0.0794	0.9051
2 white	0.0001	0.0001	0.0008	0.0813	0.6189	0.2988
3 green	0.0000	0.0000	0.0485	0.8316	0.0944	0.0255
4 yellow	0.0000	0.0040	0.9123	0.0654	0.0107	0.0077
5 blue	0.0016	0.7931	0.1948	0.0060	0.0024	0.0021
6 blue	0.8443	0.1391	0.0070	0.0044	0.0027	0.0025
7 EOS	0.3402	0.1324	0.1416	0.1438	0.1220	0.1199

The table rounds to four decimal places. A displayed 0.0000 can be a small positive weight; the calculations retain it.

13. Interpret a repeated color carefully

In the saved example, *blue* occurs at source positions 1 and 2. Read the first generated blue row. Both occurrences are visible to the scorer, but each has its own weight. We reuse the Chapter 3 checkpoint from Exercise 12. If a rerun selects another illustration without a generated repeat, choose a held-out example whose greedy output contains a color that appears at least twice in the source.

In [23]

```
def first_repeat_step(source, emitted):
    return next(
        (i for i, token in enumerate(emitted)
         if source.count(token) > 1),
        None,
    )

repeated_source = alignment_source
repeated_ids = greedy_ids
repeated_weights = greedy_weights
repeat_step = first_repeat_step(repeated_source, repeated_ids)

if repeat_step is None:
    for candidate in alignment_test:
        if len(set(candidate)) == len(candidate):
            continue
        repeat_batch = collate([candidate])
        tokens, maps = greedy_decode(
            trained_alignment, repeat_batch.source,
            repeat_batch.lengths, generation_cap,
        )
        emitted = tokens[0].tolist()
        repeat_step = first_repeat_step(candidate, emitted)
        if repeat_step is not None:
            repeated_source = candidate
            repeated_ids = emitted
            repeated_weights = maps[0]
            break
    else:
        raise RuntimeError("No generated repeated color in this run.")
repeat_color = repeated_ids[repeat_step]
repeat_row = repeated_weights[repeat_step]
print("Source", words(repeated_source))
print("Generated step", repeat_step + 1, VOCAB[repeat_color])
```

Output

```
Source blue blue yellow green white red
Generated step 5 blue
```

In [24]

```
occurrences = [
    j for j, token in enumerate(repeated_source) if token == repeat_color
]
for j in occurrences:
    print(f"Source {j + 1} ({VOCAB[repeat_color]})  {repeat_row[j]:.6f}")
combined_weight = repeat_row[occurrences].sum()
print(f"Combined weight at those positions  {combined_weight:.6f}")
peak = int(repeat_row.argmax())
print(f"Largest weight  {peak + 1} {VOCAB[repeated_source[peak]]}")
```

Output

```
Source 1 (blue)  0.001580
Source 2 (blue)  0.793124
Combined weight at those positions  0.794705
Largest weight  2 blue
```

In the saved run, the first blue comes at generated step 5. Source position 2 receives about 0.7931, while position 1 receives about 0.0016. Position 3, which contains *yellow*, still receives about 0.1948. At step 6, another blue is emitted and position 1 now has the largest weight, about 0.8443. The output vocabulary has one blue class. It contains no separate prediction class for "blue from position 1" or "blue from position 2."

The weight of 0.7931 scales the second **annotation** in a weighted sum. The annotation is a vector from the recurrent encoder. In this forward encoder, the second annotation can already contain information from the first source position. The decoder also carries information through its previous hidden and cell states. The brightest cell in a heatmap identifies the largest scalar weight on a stored vector. The rest of the weighted sum and the recurrent state also enter the computation that determines the token.

A controlled intervention. Replay greedy steps 1 through 4 with the saved weights. At the start of step 5, save the hidden and cell states, the previous token, and the encoded source memory. Measure the blue probability and the difference between the blue logit and the highest other logit for the unchanged step.

Repeat only step 5 three times, with source position 2 masked, then source position 1 masked, then a non-blue position masked as a comparison. Use the same saved state, same previous token, and same annotations in all runs. Change just one entry of the source validity mask and let softmax renormalize over the remaining positions. If removing position 2 causes a larger drop in the blue score margin, that supports a role for that annotation in this read. Even if the token remains blue, the margin can show a change in the evidence available at this step. Masking both blue positions in another run can probe whether their contributions compensate for one another.

This experiment tests access to a particular stored annotation during one read. Information from the masked position can still be present in later encoder annotations or in the saved decoder state. Removing a source token and encoding the changed sequence would ask a broader question because that edit can change later annotations and the initial decoder states. The heatmap shows the weights from the baseline run. The intervention supplies the comparison with an altered prediction.

For the separate mathematical check, only the *combined* weight on the two identical vectors matters:

$$(0.2 + 0.3)(1, 0) + 0.5(0, 1) = (0.4 + 0.1)(1, 0) + 0.5(0, 1) = (0.5, 0.5).$$

The two weight rows differ, but the contexts are equal. With the preceding hidden state, cell state, input token, and model parameters held fixed, the next decoder state and logits are therefore equal. A repeated color only guarantees the same input embedding. Different prefixes can give its two recurrent annotations different coordinates, so identical annotations are a constructed example, not an assumption about repeated colors.

In [25]

```
duplicate_notes = np.array([[1., 0.], [1., 0.], [0., 1.]])
first_map = np.array([.2, .3, .5])
second_map = np.array([.4, .1, .5])
first_read = first_map @ duplicate_notes
second_read = second_map @ duplicate_notes
np.testing.assert_allclose(first_read, [.5, .5])
np.testing.assert_allclose(first_read, second_read)
print("Two weight rows:", first_map, second_map)
print("Same context:", first_read, second_read)
```

Output

```
Two weight rows: [0.2 0.3 0.5] [0.4 0.1 0.5]
Same context: [0.5 0.5] [0.5 0.5]
```

14. Challenge: follow the learning signal

This exercise isolates one attention read with fixed annotations. The toy squared loss is not the target-token loss used to train the model. Suppress the target-step index for readability. The Kronecker delta δ_{ij} selects the term with $i = j$. Using the supplied softmax derivative,

$$\begin{aligned}\frac{\partial \mathbf{r}}{\partial e_j} &= \sum_i \mathbf{h}_i \alpha_i (\delta_{ij} - \alpha_j) \\ &= \alpha_j \mathbf{h}_j - \alpha_j \sum_i \alpha_i \mathbf{h}_i \\ &= \alpha_j (\mathbf{h}_j - \mathbf{r}).\end{aligned}$$

Increasing a score transfers weight toward its annotation and away from the current mixture. The vector $\mathbf{h}_j - \mathbf{r}$ gives that direction. If $\mathbf{g} = \partial L / \partial \mathbf{r}$, the chain rule gives the scalar score derivative

$$\frac{\partial L}{\partial e_j} = \alpha_j \, \mathbf{g} \cdot (\mathbf{h}_j - \mathbf{r}).$$

For the stated loss, $\mathbf{r} = (0.90, 0.95)$ and $\mathbf{g} = \mathbf{r} - (0, 1) = (0.90, -0.05)$. The loss is $\frac{1}{2}(0.90^2 + (-0.05)^2) = 0.40625$.

Position	$\mathbf{h}_j - \mathbf{r}$	$\mathbf{g} \cdot (\mathbf{h}_j - \mathbf{r})$	Score derivative
1	$(0.10, -0.95)$	0.1375	0.006875
2	$(-0.90, 0.05)$	-0.8125	-0.081250
3	$(0.10, 0.05)$	0.0875	0.074375

The derivatives sum to zero. Adding the same small amount to every score cannot change softmax or the loss, so the derivative along that common-shift direction must vanish. Gradient descent on the *scores* would increase the second score and decrease the other two here. In the actual model, scores are outputs of shared parameters: gradients must continue through the scorer before the optimizer changes those parameters.

If every annotation is the same vector, then $\mathbf{h}_j = \mathbf{r}$ for every j. All these score derivatives vanish. Reweighting identical information cannot change this read. This does not prevent the encoder from learning different annotations through other gradients.

In [26]

```
score_tensor = torch.tensor(np.log([.05, .10, .85]), dtype=torch.float64, requires_grad=True)
annotation_tensor = torch.tensor(annotations, dtype=torch.float64)
probabilities = torch.softmax(score_tensor, dim=0)
context_tensor = probabilities @ annotation_tensor
target_context = torch.tensor([0., 1.], dtype=torch.float64)
loss = .5 * (context_tensor - target_context).square().sum()
loss.backward()
expected_gradient = np.array([.006875, -.081250, .074375])
np.testing.assert_allclose(score_tensor.grad.numpy(), expected_gradient, atol=1e-14)

def toy_loss(score_row):
    _, result = weighted_read(score_row, annotations)
    return .5 * np.square(result - [0., 1.]).sum()

base_scores = score_tensor.detach().numpy()
epsilon = 1e-5
finite_difference = np.array([
    (toy_loss(base_scores + epsilon * direction) - toy_loss(base_scores - epsilon * direction)) / (2 * epsilon)
    for direction in np.eye(3)
])
np.testing.assert_allclose(finite_difference, expected_gradient, atol=1e-10, rtol=1e-8)
print("Toy loss:", loss.item())
print("Automatic derivatives:", score_tensor.grad.numpy())
print("Finite differences:   ", finite_difference)
print("Sum of derivatives:", score_tensor.grad.sum().item())
identical = np.tile([.4, -.2], (3, 1))
_, same_read = weighted_read(base_scores, identical)
identical_derivative = np.array([.05, .10, .85])[:, None] * (identical - same_read)
np.testing.assert_allclose(identical_derivative, 0., atol=1e-15)
print("Identical annotations give zero read derivatives.")
```

Output

```
Toy loss: 0.40625
Automatic derivatives: [ 0.006875 -0.08125   0.074375]
Finite differences:    [ 0.006875 -0.08125   0.074375]
Sum of derivatives: -2.7755575615628914e-17
Identical annotations give zero read derivatives.
```